UET NOWSHERA | ELECTRICAL ENGINEERING

LAB REPORT 10

MORE ABOUT FUNCTIONS IN C++

COURSE: Computer Programming
AUTHOR: M.Waleed Javed
REG NO: [25NWELE0749]
SECTION: [A]

1. OBJECTIVE

The primary objectives of this laboratory session are:

- To understand and apply Function Call by Value. [cite: 121]
- To understand and apply Function Call by Reference. [cite: 121]
- To understand and apply Default Values in Parameters. [cite: 121]

2. THEORETICAL BACKGROUND

Call By Value. Call by value is used when whenever the called function does not need to modify the value of the caller's original variable. [cite: 122] In this case a copy of the variable is passed to the function and when the function changes its values then it has no effect on the value of the variable in the main function. [cite: 123]

Call By Reference. If you are calling by reference it means that compiler will not create a local copy of the variable which you are referencing to. [cite: 132] It will get access to the memory where the variable saved. [cite: 133] When you are doing any operations with a referenced variable you can change the value of the variable. [cite: 134] Passing by reference is also an effective way to allow a function to return more than one value. [cite: 139] Note that & (ampersand) is called addressof operator. [cite: 124]

Default Values in Parameters. When declaring a function we can specify a default value for each of the last parameters. [cite: 141] This value will be used if the corresponding argument is left blank when calling to the function. [cite: 142] To do that, we simply have to use the assignment operator and a value for the arguments in the function declaration. [cite: 143] If a value for that parameter is not passed when the function is called, the default value is used, but if a value is specified this default value is ignored and the passed value is used instead. [cite: 144]

3. SOFTWARE USED

- IDE: DEV-C++ IDE
- Compiler: C++ Compiler (bundled with DEV-C++)

4. EXPERIMENTAL TASKS

Task 1: Function with Parameters

Declare a function called multiply that takes two integer parameters: num1 and num2. [cite: 146] Inside the function, calculate the product of the two numbers. Display the result within the function. [cite: 147] Call the function from the main program, passing two numbers, to perform the multiplication. [cite: 148]

```
#include <iostream>
using namespace std;

// Function to multiply two parameters and display
void multiply(int num1, int num2) {
    int product = num1 * num2;
    cout << "The product of " << num1 << " and " << num2 << " is: " <<
product << endl;
}

int main() {
    // Function call with two arguments
    multiply(7, 8);
    return 0;
}
```

Task 2: Function with Return Value

Declare a function called getSquare that takes an integer parameter: number. [cite: 149] Inside the function, calculate the square of the number. Return the result. [cite: 150] Call the function from the main program, passing a number, and display the square. [cite: 151]

```
#include <iostream>
using namespace std;

// Function that returns the square of the parameter
int getSquare(int number) {
    return number * number;
}

int main() {
    int val = 9;
```

```
    // Function call passing a number, capturing the return
    int sq = getSquare(val);

    cout << "The square of " << val << " is: " << sq << endl;
    return 0;
}
```

5. CONCLUSION

This lab expanded upon functions in C++, demonstrating the critical differences between call by value and call by reference. By utilizing call by reference, programs can manipulate original variables directly and return multiple outcomes from a single function. Additionally, implementing default parameters simplifies function calls by providing fallback values. These techniques allow for more flexible, efficient, and sophisticated C++ programming.